

Rapport de synthèse — 31 juillet au 2 août 2026

Période : vendredi 31 juillet, samedi 1er août, dimanche 2 août 2026

Branches : Bug_Streising, LUT_calibration_all_wavelength, TG-FROG_implementation

*Rapports détaillés sous-jacents : rapport_debogage_2026-07-31_bug_streising_pixis_camera.md,
rapport_debogage_2026-08-01.pdf,
rapport_debogage_2026-08-02_LUT_calibration_multi_longueur_onde.md*

Résumé exécutif

Trois journées de travail avec un fil conducteur : vendredi a stabilisé la chaîne d'acquisition sans résoudre son symptôme de départ, samedi a résolu ce symptôme en mettant au jour toute la chaîne de causes qui le masquait, et dimanche a ouvert deux nouveaux fronts — la calibration spectrale du SLM et la caractérisation d'impulsion par TG-FROG.

- Vendredi — 8 bugs corrigés (dérive de phase, double contrôle du cryostat, perte de 99 % du signal Pixis, plantages au démarrage), 5 ajouts (onglet caméra, visualiseur de log). La caméra Streising refusait toujours de s'initialiser (*Getting DMA buffer failed*) à la fin de la session.

- Samedi — 13 correctifs sur la chaîne d'acquisition complète, dont deux régressions introduites puis corrigées dans la même session. Le symptôme de vendredi s'est révélé être la conséquence de cinq causes empilées, pas une seule ; la dernière (miroir de sortie du spectrographe mal dirigé) n'avait rien à voir avec le DMA. Résolu et confirmé avec un laser hélium-néon sur les deux caméras.

- Dimanche — Correction de la calibration LUT du SLM pour tenir compte de la largeur spectrale du pulse (physique de diffraction), puis implémentation d'une première capacité de caractérisation d'impulsion par TG-FROG (acquisition, simulateur, retrieval), avec plusieurs bugs trouvés et corrigés en testant.

1. Corrections de bugs

1.1 Vendredi 31 juillet — branche Bug_Streising

BeamExplorer — la phase optimale s'additionnait à chaque clic sur Apply. La ligne « Current » du tableau de phase s'affiche toujours en absolu, mais `set_phase_manually()` et `update_beam()` réécrivaient cette valeur sans préciser de mode, retombant sur le mode relatif par défaut — qui traite la valeur reçue comme un delta et l'additionne à la phase optimale. Une valeur déjà absolue se faisait donc réadditionner l'optimale à chaque apply, faisant dériver la phase envoyée au SLM. Corrigé en forçant `mode='absolute'` sur les deux écritures. (`BeamExplorer.py`)

Cryostat — double contrôle entre la banderole générique et l'onglet dédié. Plusieurs paramètres du cryostat étaient marqués éditables côté driver et donc modifiables depuis la banderole latérale, en plus de l'onglet Cryostat dédié — deux chemins d'écriture vers le même matériel. La construction de l'arbre de paramètres force maintenant le read-only pour le device `cryostat`, peu importe le flag du driver. `CryoDemo.py` a aussi été réécrit pour implémenter les méthodes que `CryostatTab` appelait déjà mais qui n'existaient pas côté démo. (`main.py`, `CryoDemo.py`)

Pixis — la caméra ne lisait qu'une ligne sur 252-256 (perte de ~99 % du signal). La ROI était figée à une seule ligne du capteur, décrite en commentaire comme faisant « agir la caméra comme un tableau 1D » — mais sur un spectrographe, l'axe vertical du capteur est la position le long de la fente d'entrée, pas une donnée redondante.

Ajout de `set_roi()/set_binned_roi()/set_full_frame()/get_roi()`, avec un vrai binning sur puce (le bruit de lecture n'est payé qu'une fois). (`Pixis.py`)

Onglet Camera — `TypeError` au démarrage. Le spectromètre actif par défaut (Stresing) a un signal `sendSpectrum` à un seul argument, celui de Pixis en a deux ; la connexion se faisait sans vérifier la compatibilité. Corrigée pour ne se faire que si le spectromètre expose `set_binned_roi`. (`main.py`)

Onglet Camera — vue figée hors-champ après un changement de mode. Aucun réajustement automatique du cadrage lors d'un changement de forme de trame. Ajout d'un ajustement automatique au changement de forme, plus un bouton manuel « Fit view ». (`CameraDisplay.py`)

Monochromateur absent — plantage total au démarrage. `SpectraPro2300i` était le seul appareil de `load_instruments()` instancié sans `try/except` ; son port série s'ouvre immédiatement, donc son absence faisait planter toute la fonction. Ajout d'un driver démo (`SpectraPro2300iDemo.py`) et du même pattern `try/except` que les autres appareils.

Stresing — buffer DMA jamais configuré. `dma_buffer_size_in_scans` restait à 0 par défaut ; le fichier de configuration réellement chargé au démarrage s'est en plus révélé externe au dépôt et désynchronisé. Corrigé avec une valeur de repli (1000) si la clé est absente du fichier. (`StresingDriver.py`)

Stresing — `BaseException` non capturée, plantage total (8 occurrences). Les 8 vérifications d'erreur du DLL levaient `BaseException` au lieu d'une sous-classe d'`Exception` — `except Exception:` ne les capturait donc pas. Changé en `RuntimeError`. Ce bug préexistait mais ne s'était jamais manifesté aussi violemment avant le correctif du buffer DMA. (`StresingDriver.py`)

1.2 Samedi 1er août — branche `Bug_Streising` (suite)

La session est partie du symptôme resté ouvert vendredi (*Getting DMA buffer failed*) et a découvert, en corrigeant successivement, quatre autres causes masquées derrière : l'interface gelait sur une lecture matérielle bloquante exécutée dans le thread graphique ; la carte attendait un trigger externe absent, sans échéance ; une `IndexError` silencieuse empêchait l'émission du signal avant même l'affichage ; et le miroir de sortie du spectrographe dirigeait la lumière vers le mauvais port de caméra. Treize correctifs au total, regroupés par sujet ci-dessous.

Les spectres n'atteignaient jamais le graphique (le bug le plus grave et le plus ancien de la session). `concatenate_data()` lisait `queue[-1]` sur une file de paramètres toujours vide, car `update_parameter()` n'était appelée nulle part dans le code — la lecture levait une `IndexError` à chaque spectre, avant l'émission du signal vers le graphique. Conséquence : aucun spectre ne s'est jamais affiché, quel que soit le spectromètre, tant que ce code existait, sans la moindre erreur visible dans le journal. Un second défaut du même commit peuplait le graphique au démarrage d'une gaussienne aléatoire (177-884 nm) qui imposait cette plage aux axes, comprimant un spectre réel de 48 nm dans une bande illisible. (`DataHandling.py`, `SpectrometerPlot.py`)

Gel complet de l'interface. `acquire_measurement()` appelait la lecture matérielle bloquante directement depuis le thread graphique — l'interface restait figée aussi longtemps que la caméra mettait à répondre, potentiellement indéfiniment si la carte attendait un trigger absent. Diagnostiqué avec `py-spy` (pile d'appels d'un processus vivant). (`main.py`)

Stresing — worker court-circuité, modes de trigger inutilisables, aucune échéance. `get_intensities()` appelait la méthode du worker en non-liée, en lui passant la caméra comme `self` — le signal `sendSpectrum` n'était donc jamais émis, et la boucle d'attente du worker aurait tourné indéfiniment si elle avait été atteinte. Les registres de trigger (`sti/bti`) partagent une numérotation mais divergent au-delà de 4, avec un temporisateur qui ne s'applique que dans un seul mode — exposés en 8 champs numériques indépendants, ils étaient inutilisables sans documentation. Tout attente d'un signal externe passe maintenant par un sondage avec échéance explicite plutôt qu'un appel bloquant sans limite. (`Stresing.py`, `StresingDriver.py`)

SpectraPro — le miroir de sortie ne bougeait pas. La Stresing est physiquement sur le port **side**, la Pixis sur le port **front** du spectrographe : la caméra sur l'autre port ne recevait rien. Le pilote envoyait `MIRROR`, une commande que le contrôleur accepte en répondant `ok` sans rien faire (vérifié par dialogue série brut) ; les commandes correctes sont `EXIT-MIRROR` puis `FRONT/SIDE`. Deux défauts additionnels dans le même fichier : le nombre de réseaux de la tourelle était mal déduit (4 au lieu de 3, un réseau fantôme inventé), et la boucle de lecture série n'avait aucune porte de sortie face à un contrôleur muet. (`SpectraPro2300i.py`)

La soustraction de fond corrompait silencieusement les spectres — le défaut le plus dangereux de la session, car il produit des chiffres faux plutôt qu'une panne visible. Le tableau de fond était alloué avec `np.empty` (contenu mémoire arbitraire, non initialisé) et jamais réellement rempli par une mesure de fond. Cocher la correction soustrayait donc des valeurs arbitraires de chaque spectre sans la moindre erreur. Le tableau est maintenant mis à zéro et protégé par un indicateur ; la correction est ignorée avec avertissement tant qu'aucun fond valide n'a été acquis ou chargé. (`DataHandling.py`, `main.py`, `MeasurementClasses.py`)

Paramètres matériels jamais enregistrés, interface bloquée en cas d'échec, sauvegarde en course. `update_parameter()` attendait un format jamais fourni et restait vide — aucun réglage n'était enregistré à côté des données. `measurement_busy` n'était libéré que sur un succès complet, laissant l'interface bloquée sur **devices are busy** après toute exception. `save_data()` copiait le fichier après un délai fixe d'une demi-seconde au lieu d'attendre la fin réelle de l'écriture, risquant de copier un fichier à moitié écrit sur un disque lent. Les trois corrigés ensemble. (`DataHandling.py`, `main.py`, `MeasurementClasses.py`)

Pixis pilotée par deux threads à la fois. PICam refuse de démarrer une acquisition déjà en cours ; `get_intensities()` (thread de mesure) et `set_roi()` (thread graphique) démarraient/arrêtaient la caméra indépendamment, provoquant `AcquisitionInProgress` si une région était appliquée pendant une vue en direct. Les deux passent maintenant par un verrou réentrant. (`Pixis.py`)

Deux régressions introduites par ces correctifs, puis corrigées dans la même session. Le délai d'attente ajouté pour les modes de trigger appelait `DLLStopMeasurement` — une fonction qui n'existe pas dans ce DLL — entourée d'un `except AttributeError` qui avalait silencieusement l'erreur ; chaque expiration abandonnait donc sans jamais arrêter la carte, cassant la mesure suivante. Le correctif de cette première régression (retirer l'appel bloquant au profit du chemin non bloquant) en a produit une seconde : `DLLStartMeasurement_nonblocking` renvoie 976 « An unknown error occurred » sur cette carte, code de retour jamais vérifié — plus aucune mesure ne démarrait. Solution retenue : garder l'appel bloquant, seul fonctionnel, mais interroger d'abord la carte pour savoir si un trigger externe est réellement arrivé avant de s'y engager. Ces deux régressions ont été commises en corrigeant précisément ce type de défaut — la leçon retenue est qu'un DLL propriétaire dont les erreurs sont des codes de retour exige une discipline explicite, sans quoi la panne redevient invisible par construction. (`Stresing.py`, `StresingDriver.py`)

Réglages caméra et calibration Pixis. La température du capteur Pixis était déclarée modifiable dans l'arbre de paramètres alors que rien ne la règle (seul le worker la met à jour) — elle ne se rafraîchissait donc jamais après le démarrage. Le temps de pose était converti par `int()` avant d'atteindre la caméra, écrasant à zéro toute valeur sous la milliseconde — précisément la plage utile sur ce montage. (`Pixis.py`)

Le spectre binné semblait tomber à zéro après un spectre plein cadre. Mesuré sur le banc : le plein cadre somme 252 lignes en Python sans écrêtage (jusqu'à 16,5 millions de comptes), le binné sature sur la puce à 65 535 — un facteur ~250 entre les deux échelles. Comme le graphique accumule les courbes sans les distinguer, celle du plein cadre s'appropriait les axes. Appliquer une zone de lecture vide maintenant les deux graphes. (`Pixis.py`, `CameraDisplay.py`, `main.py`)

1.3 Dimanche 2 août — branches `LUT_calibration_all_wavelength` et `TG-FROG_implementation`

LUT SLM — troncature au lieu d'arrondi. `digital_image.astype(dtype)` tronque vers zéro plutôt que d'arrondir — un biais systématique toujours vers le bas sur chaque valeur de greyscale. Trouvé par un test qui attendait 154 et obtenait 153. Corrigé en `np.round(clipped).astype(dtype)` dans le pilote réel et le pilote démo. (`SLM.py`, `SLMDemo.py`)

TG-FROG — double conversion du facteur jacobien λ^2 en mode démo. Le premier test en boucle fermée du simulateur (impulsion connue, GDD/TOD attendus) donnait un GDD retrouvé proche de zéro au lieu de la valeur simulée. Cause détaillée en §2.4 (physique). Corrigé en appliquant le jacobien inverse dans la génération de la trace synthétique. (`TGFROGClasses.py`, `samples/retrieval/tgfrog_retrieval.py`)

TG-FROG — « Simulate trace » ne faisait rien sur une application fraîche. Le mode démo exigeait trois faisceaux distincts déjà sélectionnés dans les menus déroulants, alors qu'il ne touche jamais les objets Beam réels — sur une application sans faisceau encore défini, les trois menus vides déclenchaient silencieusement le garde-fou (un simple `print()` console, invisible dans l'interface). Corrigé en repliant sur des noms de substitution en mode démo uniquement, et en affichant l'erreur dans l'étiquette de statut plutôt que sur la console. (`main.py`)

TG-FROG — le correctif précédent cassait le garde-fou du mode réel. Les noms de substitution introduits ci-dessus étant déjà distincts entre eux, ils contournaient sans le vouloir la vérification « trois faisceaux différents » censée bloquer une acquisition matérielle réelle sans faisceaux sélectionnés. Trouvé en testant explicitement ce second scénario ; corrigé en séparant clairement les deux chemins (repli en mode démo, validation stricte non-vide + distincte en mode réel). (`main.py`)

TG-FROG — plantage répété de `concatenate_data` en mode démo. Le mode démo synthétise son propre axe de longueur d'onde, de taille différente du spectromètre configuré au démarrage de l'application ; l'envoyer vers le tampon général de spectres (qui suppose une taille fixe) faisait échouer `np.c_[...]` à chaque point de délai. Corrigé en ne connectant pas ce signal pour le TG-FROG — la trace 2D complète étant de toute façon déjà capturée par un signal dédié. (`main.py`)

TG-FROG — le graphique de trace ne se recadrait jamais. Sans appel explicite à l'auto-ajustement de la vue, le graphique gardait le niveau de zoom de son tout premier point (2 points de délai) et ne s'ajustait plus ensuite, laissant la trace réelle écrasée dans un coin d'un cadre majoritairement noir. Trouvé sur une capture d'écran fournie par l'utilisateur. Corrigé par un appel à l'auto-ajustement de la vue à chaque mise à jour. (`GUI/PulseCharacterization.py`)

2. Implémentations et ajouts

2.1 Vendredi — Onglet Camera, visualiseur de log, fondations démo

Onglet Camera : vue 2D en direct de la trame caméra avec profil vertical, détection automatique de bande de signal, bascule plein cadre/binné. Alimenté directement par le worker caméra, hors de la chaîne `DataHandling` — c'est une vue d'alignement, pas une donnée de mesure. (`CameraDisplay.py`)

Visualiseur de log (File > View Log) : fenêtre non-modale affichant `main.log` en direct, case « Follow » pour le défilement automatique. A directement servi à diagnostiquer les bugs Stresing du lendemain. (`LogViewer.py`)

Script de diagnostic Pixis autonome, driver démo du monochromateur, fusion de deux mois de travail parallèle sur l'intégration Pixis/monochromateur/cryostat.

2.2 Samedi — Panneau d'acquisition unifié

Nouveau panneau `AcquisitionSettings`, remplaçant l'édition directe des registres du contrôleur Stresing. Il expose les trois configurations de déclenchement réellement utilisées sur le montage — la carte sur son propre

temporisateur interne, une lecture par impulsion laser externe, des blocs cadencés par un hacheur mécanique — et déduit automatiquement les registres correspondants plutôt que de les exposer en 8 champs numériques indépendants et sans lien apparent entre eux. Les intervalles se saisissent avec leur unité et affichent fréquence et durée totale déduites ; les réglages qu'un mode donné n'utilise pas sont masqués plutôt que laissés visibles sans effet. Le panneau porte aussi la configuration du monochromateur (longueur d'onde centrale, réseau, port de sortie nommé d'après la caméra qu'il alimente) et les réglages propres à chaque caméra (temps de pose et moyennage pour Pixis, gain ADC pour Stresing), construits directement depuis le dictionnaire de paramètres du pilote plutôt que codés en dur par caméra. (`AcquisitionSettings.py`, `main.py`, `Pixis.py`)

2.3 Dimanche — Calibration LUT du SLM dépendante de la longueur d'onde

Contexte physique. Le SLM ne module pas directement une phase : il façonne par diffraction (méthode Turner/Nelson), un réseau de phase en dents de scie dont l'ordre 1 porte la phase désirée via un décalage latéral. Pour une amplitude réelle de réseau A ($A=1$ correspondant à une excursion de phase de exactement 2π), l'amplitude de diffraction à l'ordre 1 est

Coefficient d'ordre 1 : $c_1 = \text{sinc}(A-1) \cdot e^{i\pi(A-1)} \cdot e^{-i\phi}$, où $\text{sinc}(x) = \sin(\pi x)/(\pi x)$

ce qui donne une efficacité de diffraction $\eta_1 = \text{sinc}(A-1)^2$ (maximale à $A=1$) et un terme d'erreur de phase $\pi(A-1)$ qui s'ajoute à $-\phi$.

Le SLM ne contrôle pas la phase directement mais une tension, qui pilote la biréfringence Δn du cristal liquide et donc une retardance $\Gamma(g) = \Delta n(g) \cdot d$ — une longueur physique indépendante de la couleur de la lumière. La phase vue à une longueur d'onde λ donnée est $\phi = 2\pi\Gamma/\lambda$. La calibration existante (`Generate_LUT_PhasetoGreyscale`) mesure $\phi(g)$ à une seule longueur d'onde de calibration λ_{cal} et linéarise la LUT en conséquence, ce qui rend automatiquement Γ linéaire en greyscale : $\Gamma(g) = \lambda_{\text{cal}} \cdot g / 1023$ — la calibration existante calibre en réalité la retardance, la phase-à- λ_{cal} n'en étant qu'une lecture particulière.

Le bug caché. Le code actuel calcule le greyscale requis à partir de la phase désirée en supposant λ_{cal} partout sur le panneau, peu importe la colonne réelle. Si la lumière à cette colonne est à $\lambda(x) \neq \lambda_{\text{cal}}$, la phase réellement produite devient $\phi_{\text{réel}} = \phi_{\text{désirée}} \cdot \lambda_{\text{cal}} / \lambda(x)$ — c'est-à-dire une erreur d'amplitude de réseau $A = \lambda_{\text{cal}} / \lambda(x)$, pas une erreur de phase arbitraire.

Quantification ($\lambda_{\text{cal}} = 785$ nm) : de 650 à 950 nm, l'efficacité de diffraction chute jusqu'à 86,6 % (perte de 13,4 % en bord de bande bleue) et 90,5 % côté rouge. Décomposé en ordres de dispersion sur cette même plage, le terme de phase $\pi(A-1)$ reste presque exactement un délai de groupe pur (+1,3 fs) tant qu'on néglige la dispersion de $\Delta n(\lambda)$; en incluant une dispersion réaliste du cristal liquide, un résidu de GDD apparaît (+0,29 fs²) — négligeable devant les centaines à milliers de fs² typiquement corrigés en pratique. Conclusion physique : calibrer à une seule longueur d'onde ne dégrade quasiment pas la compression ; le vrai coût est en efficacité de diffraction, qui agit comme un filtre spectral et allonge légèrement l'impulsion effective — un effet qui devient significatif avec la largeur de bande beaucoup plus grande attendue du NOPA en construction, alors qu'il est négligeable avec l'OPA actuel à bande étroite.

Correction implémentée. Puisque g et Γ sont déjà liés linéairement par la calibration existante, on inverse simplement pour obtenir le greyscale corrigé par colonne :

$$g_{\text{corrigé}}(x) = g_{\text{naïf}}(x) \cdot \lambda(x) / \lambda_{\text{cal}}$$

Une multiplication par colonne appliquée juste avant l'écriture au SLM, sans aucune nouvelle mesure de calibration. Câblée dans `SLMWorker.apply_wavelength_correction()`, avec avertissement journalisé (limité en fréquence) si la correction sature — signe qu'une phase demandée dépasse ce que le panneau peut produire à cette longueur d'onde locale. Le nom du fichier `.lut` chargé ou généré s'affiche maintenant en permanence dans la barre de statut ; volontairement, aucune extraction automatique de la longueur d'onde depuis ce nom de fichier n'a été implémentée, jugée trop fragile — l'utilisateur lit le nom lui-même. (`SLM.py`,

SLMDemo.py, main.py, Calibration_Classes.py)

2.4 Dimanche — Caractérisation d'impulsion par TG-FROG

Objectif. Ajouter une capacité de mesure de la phase spectrale de l'impulsion (durée, GDD, TOD) par transient-grating FROG, en réutilisant le montage boxcar existant plutôt qu'un montage dédié.

Géométrie et physique. Deux des trois faisceaux du montage (les faisceaux « réseau ») restent à leur propre phase courante, à délai relatif nul, et interfèrent dans un milieu non résonant pour y écrire un réseau transitoire (spatial, d'indice de réfraction). Le troisième faisceau (la « sonde ») est balayé en délai via le SLM et diffracte sur ce réseau dans la direction accordée en phase du montage boxcar. Pour un délai de sonde τ , le champ du signal TG s'écrit

$$E_{\text{signal}}(t, \tau) = E_A^*(t - \tau) \cdot E_B(t) \cdot E_C(t)$$

où la sonde apparaît conjuguée — cette convention de signe correspond exactement au processus PNPS(pulse, "frog", "tg") de la librairie `pypret` utilisée pour le retrieval. Le délai est imposé numériquement par le SLM (pas de platine mécanique), avec une amplitude accessible de l'ordre de plusieurs picosecondes pour un besoin de quelques centaines de femtosecondes.

Contrainte de couverture spectrale. Le signal TG est proportionnel à $|E|^2 E$, donc racine(3) plus court en temps que l'impulsion fondamentale — et racine(3) plus large en fréquence. Combinée à la relation limite Fourier pour une gaussienne, la largeur spectrale nécessaire du signal suit approximativement

$$\Delta\lambda_{\text{signal}}[\text{nm}] \approx 1625 / \Delta t[\text{fs}]$$

Pour une impulsion de 20 fs, cela demande déjà ~81 nm de couverture ; pour 10 fs, ~163 nm — contre ~46-53 nm disponibles sur le réseau 1200 traits/mm actuellement installé sur le monochromateur. Un test en boucle fermée du simulateur (impulsion de 40 fs, fenêtre de 50 nm, imitant le réseau et l'OPA actuels) a confirmé empiriquement que la seule largeur à mi-hauteur ne suffit pas : le GDD retrouvé était complètement faux (mauvais signe, mauvais ordre de grandeur) malgré une largeur nominale suffisante — une marge de plusieurs fois la largeur à mi-hauteur est nécessaire en pratique, pas juste son égalité. Cette question de couverture spectrale et le lien entre durée d'impulsion actuellement disponible (OPA) contre future (NOPA) et le réseau du monochromateur reste le facteur bloquant pour une mesure fiable sur le matériel réel.

Ce qui a été implémenté (branche TG-FROG_implementation):

- `TGFROGMeasurement` (`src/measurements/TGFROGClasses.py`) : classe de mesure balayant le délai de la sonde, calquée sur le patron déjà établi de `DelayCalibrationMeasurement` pour l'application de phase.

- Un mode démo qui génère une vraie trace synthétique via le simulateur direct de `pypret` (pas un module vide), à partir d'une impulsion gaussienne de GDD/TOD connus — ce qui a permis de découvrir le bug du jacobien (§1.3) avant tout contact avec du matériel réel.

- Onglet « Pulse characterization » (`src/GUI/PulseCharacterization.py`) : sélection des trois rôles de faisceau, paramètres de balayage, et un panneau simulateur interactif exposant durée d'impulsion, GDD, TOD, niveau de bruit et largeur de fenêtre spectrale directement dans l'interface — avec une estimation en direct de la largeur spectrale requise pour la durée entrée, permettant d'explorer la question OPA/NOPA sans écrire de code.

- `TGFROGRetrievalWorker` : le retrieval (algorithme COPRA via `pypret`, 5 redémarrages indépendants pour limiter le risque de minimum local sur une trace bruitée) tourne maintenant directement depuis un bouton « Run retrieval » dans l'onglet, dans un thread séparé, avec un panneau affichant l'erreur de trace et le GD/GDD/TOD retrouvés. Un script en ligne de commande équivalent (`samples/retrieval/tgfrog_retrieval.py`) reste disponible pour rejouer une trace exportée avec d'autres réglages.

- `pypret` (aucune version publiée sur PyPI) installé en mode éditable depuis un clone épinglé à un commit précis, dans un dossier `../pypret` en dehors du dépôt COLBERTo.

État : fonctionnel et testé de bout en bout hors-écran (mode démo uniquement — aucun test sur signal TG réel). Voir §3 pour les limites connues.

3. Bug report — problèmes identifiés, non corrigés

#	Sujet	Origine	Description
3.1	Fond des scans de chirp — usage ambigu	Samedi	<code>AcquireBackground</code> envoie toujours son résultat au seul dictionnaire de calibration. Sert-il à la soustraction générale ou uniquement à la calibration ? Les deux usages sont légitimes, le code ne tranche pas.
3.2	Calibration Pixis — deux jeux de paramètres divergents	Samedi	Le fichier d'initialisation calcule un jeu de paramètres ajustés jamais utilisé ; le dictionnaire réellement employé porte des valeurs qui ressemblent à des valeurs par défaut, notamment sur le pas du réseau. Lequel est correct reste à trancher en vérifiant la position du pic d'un laser connu.
3.3	Modes laser/hacheur non validés en conditions réelles	Samedi	Le mode synchronisé abandonne proprement sans trigger (vérifié), mais aucune acquisition n'a été faite avec un laser réellement branché ; le mode hacheur est construit d'après la documentation du pilote uniquement.
3.4	Tampon disque au-delà de 50 spectres non testé	Samedi	Une sauvegarde courte est validée de bout en bout ; le vidage périodique du tampon vers le fichier temporaire sur un enregistrement continu de plusieurs centaines de spectres ne l'est pas.
3.5	Relecture d'un fichier sauvegardé non testée	Samedi	L'écriture est vérifiée ; le cycle complet sauvegarder -> recharger -> comparer ne l'a pas été.

#	Sujet	Origine	Description
3.6	<code>save_parameter()</code> — fichier séparé, jamais appelé	Samedi	Écrit un fichier de paramètres distinct, orphelin dans le code. À garder ou retirer, maintenant que les paramètres accompagnent déjà les spectres.
3.7	Longueurs d'onde sans monochromateur	Samedi	La Stresing renvoie alors le nombre de pixels au lieu d'un tableau d'indices, malgré ce qu'annonce son message d'avertissement. Sans effet tant qu'un monochromateur reste attaché, ce qui est le cas actuellement.
3.8	LUT — dispersion $\Delta n(\lambda)$ du cristal liquide estimée, pas mesurée	Dimanche	La valeur utilisée dans le calcul de quantification (§2.3) est représentative d'un nématique typique, pas mesurée sur le panneau actuel. Le protocole à 3 points décrit dans le rapport LUT permettrait de la mesurer réellement, mais n'a pas été implémenté.
3.9	LUT — validation sur matériel réel manquante	Dimanche	Tout le raisonnement du §2.3 repose sur la physique et des tests logiciels hors-écran ; aucune confirmation sur banc.
3.10	LUT — le spinbox de longueur d'onde de calibration ne se restaure pas après chargement	Dimanche	Limite préexistante, pas introduite cette session : après le chargement d'une calibration sauvegardée, la valeur affichée peut ne plus refléter λ_{cal} réelle tant qu'elle n'est pas re-confirmée manuellement.
3.11	TG-FROG — aucune validation sur signal réel	Dimanche	L'implémentation complète (acquisition, simulateur, retrieval) n'a été testée qu'en mode démo hors-écran ; aucun signal TG réel n'a encore été mesuré.

#	Sujet	Origine	Description
3.12	TG-FROG — couverture spectrale du réseau actuel probablement insuffisante	Dimanche	Voir §2.4 : le réseau 1200 traits/mm (~46-53 nm) est vraisemblablement trop étroit pour un retrieval fiable avec la durée d'impulsion de l'OPA actuel, et certainement pour le NOPA à venir — reste à mesurer la durée réelle de l'OPA pour trancher précisément. Pas un bug logiciel, mais une contrainte matérielle qui bloque la validation de 3.11 telle quelle.
3.13	TG-FROG — retrieval sensible aux minima locaux sous bruit	Dimanche	COPRA (via <code>pypret</code>) peut converger vers un minimum local sur une trace bruitée en un seul essai ; le multi-démarrage (5 essais indépendants, garde la meilleure erreur de trace) réduit le risque sans l'éliminer. Comportement inhérent à l'algorithme, documenté dans la littérature, pas spécifique à cette implémentation.
3.14	<code>main.py</code> — code non protégé par <code>if __name__ == '__main__':</code>	Dimanche	Les dernières lignes du fichier (<code>app = QApplication(...); window = MainInterface(); app.exec_()</code>) s'exécutent à l'import, pas seulement en lancement direct. Sans effet sur l'usage normal (<code>python main.py</code>), mais empêche d'importer le module pour des tests automatisés sans contourner l'appel bloquant. Trouvé en testant l'onglet Pulse characterization hors-écran, non corrigé (hors sujet de cette session).